Proof-Writing Style Guide

BCS 402 | Computability & Complexity

Reference: Sipser's presentations linked from the top of every page in the course learning tool.

The Sipser style in three habits

Every proof, reduction and Turing-machine construction in this course should follow **Sipser's style** — the same style used in the lecture material accessible from the *top of every page* in your learning tool. The habits to internalise are:

1. **Number every step.** Don't write paragraphs; write *Step 1*, *Step 2*, *Step 3*, ...
2. **Define TMs by "On input x:".** Whenever you introduce a new machine, name it and list its actions as numbered steps.
3. **Close with a one-sentence conclusion.** Always end with *"Therefore L is ..."* followed by ■.

The sections below pair a *template* (the shape of a proof) with a *worked example* that puts the template to work. The point is the shape: the same shape carries over to any proof of the same kind.

---

1. Diagonalisation for undecidability

Template — Sipser-style diagonal contradiction proof (4 steps)

Step 1. Assume a decider H for the target language exists.

Step 2. Construct a diagonal TM D that uses H as a subroutine and produces the *opposite* of H's prediction.

Step 3. Feed D its own description $\langle D \rangle$ and derive a contradiction in *both* branches.

Step 4. Conclude: "Therefore the language is undecidable. ■"

Worked example — The language $L_{\text{SA}} = \{\langle M\rangle : M \text{ accepts its own description } \langle M\rangle\}$ is undecidable.

Step 1. Suppose, for contradiction, that a TM H decides L_{SA}:

$$H(\langle M\rangle) = \textsc{accept} \text{ iff } M \text{ accepts } \langle M\rangle, \quad H \text{ always halts.}$$

Step 2. Construct a TM D:
On input $\langle M\rangle$:

1. Run H on $\langle M\rangle$.
2. If H accepts, REJECT; if H rejects, ACCEPT.

Equivalently, D accepts $\langle M\rangle$ iff M does *not* accept $\langle M\rangle$.

Step 3. Consider $D(\langle D\rangle)$.

- If D accepts $\langle D\rangle$, then by Step 2 we have $H(\langle D\rangle) = \textsc{reject}$, which means D does *not* accept $\langle D\rangle$ — contradiction.
- If D rejects $\langle D\rangle$, then $H(\langle D\rangle) = \textsc{accept}$, which means D *does* accept $\langle D\rangle$ — contradiction.

Step 4. Both branches contradict, so H cannot exist. Therefore L_{SA} is undecidable. ■

2. Non-recognisability via the bridge theorem

Template — Show L is not Turing-recognisable

Step 1. Assume L is T-recognisable; let R be a recogniser.

Step 2. Show that the *complement* $\overline{L}$ is also T-recognisable.

Step 3. Two recognisers in parallel give a *decider* for L.

Step 4. Cite a known undecidability fact about L to derive a contradiction.

Worked example — The language NEVER-HALTS $= \{\langle M\rangle : M$ does not halt on any input$\}$ is not Turing-recognisable.

Step 1. Suppose NEVER-HALTS is T-recognisable; let R be a recogniser.

Step 2. The complement $\overline{\text{NEVER-HALTS}} = \{\langle M\rangle : \exists x$ s.t. M halts on $x\}$ is T-recognisable by dovetailing:

On input $\langle M\rangle$:

1. For increasing $t = 1, 2, 3, \ldots$, simulate M on every input of length $\leq t$ for t steps.
2. If some simulation halts, ACCEPT.

Step 3. Run R and the dovetail recogniser in parallel on $\langle M\rangle$. Exactly one will eventually accept, so the parallel machine is a *decider* for NEVER-HALTS.

Step 4. But "M does not halt on any input" is a non-trivial semantic property of $L(M)$, so by Rice's theorem NEVER-HALTS is undecidable. Contradiction. Therefore NEVER-HALTS is not T-recognisable. ■

3. Reduction $A_{\text{TM}} \leq L$ for undecidability

Template — Sipser-style reduction proof

Step 1. Assume R decides L.

Step 2. Build a decider S for A_{TM} using R as a subroutine. State S as "On input $\langle M, w\rangle$:" followed by numbered actions, one of which builds an auxiliary TM M' and runs R on $\langle M'\rangle$.

Step 3. Prove correctness across *every* relevant case for M on w.

Step 4. Close: "S would decide A_{TM} — contradiction. Therefore L is undecidable."

Worked example — Let TOTAL-TM $= \{\langle M\rangle : M$ halts on every input$\}$. Show TOTAL-TM is undecidable.

We give a reduction $A_{\text{TM}} \leq$ TOTAL-TM.

Step 1. Assume R decides TOTAL-TM.

Step 2. Construct a decider S for A_{TM}:
On input $\langle M, w\rangle$*:*

1. Build a new TM M'':

   On input x*:*

   1. Simulate M on w for $|x|$ steps.
   2. If M accepts in those $|x|$ steps, enter an infinite loop.
   3. Otherwise, halt.
2. Run R on $\langle M''\rangle$. If R rejects, ACCEPT; else REJECT.

Step 3. Correctness:

- If M accepts w (say in t steps): for every input x of length $\geq t$, M'' enters the infinite loop. So M'' is not total, $\langle M''\rangle \notin$ TOTAL-TM, R rejects, S ACCEPTs.
- If M does not accept w: the loop branch is never entered, so M'' always reaches step 3 and halts. M'' is total, R accepts, S REJECTs.

Step 4. S would decide A_{TM}. But A_{TM} is undecidable. Contradiction. Therefore TOTAL-TM is undecidable. ■

4. Rice's theorem

Template — Apply Rice's theorem to property P

Step 1. State Rice's theorem precisely.

Step 2. Verify (A) SEMANTIC: the property depends only on $L(M)$, not on the description $\langle M\rangle$.

Step 3. Verify (B) NON-TRIVIAL: name one L_{yes} with the property and one L_{no} without.

Step 4. Conclude by Rice.

Worked example — Show that $L_{\text{INF}} = \{\langle M\rangle : L(M) \text{ is infinite}\}$ is undecidable.

Step 1 (statement). Rice's theorem: for every property P of T-recognisable languages that is (A) semantic and (B) non-trivial, the language $\{\langle M\rangle : L(M) \text{ has } P\}$ is undecidable.

Step 2 (semantic). Infiniteness of $L(M)$ depends only on the *set* $L(M)$, not on the encoding $\langle M\rangle$: if $L(M_1) = L(M_2)$, both are simultaneously infinite or finite. ✓

Step 3 (non-trivial). • Witness $L_{\text{yes}} = \Sigma^*$: infinite, satisfies the property.

• Witness $L_{\text{no}} = \emptyset$: finite, fails the property.

Step 4. By Rice's theorem, L_{INF} is undecidable. ■

5. Polynomial-time verifier

Template — Show $L \in$ NP given a certificate c

Step 1. Describe the verifier V as "On input (x, c):" with every check it performs and the accept condition.

Step 2. Give the running time of V in big-O as a function of $|x|$.

Step 3. Conclude: "Therefore $L \in$ NP."

Worked example — Show that SubsetSum $= \{\langle S, t\rangle : \exists S' \subseteq S$ with $\sum S' = t\}$ is in NP, given the certificate $c = (a_{i_1}, \ldots, a_{i_m})$ — the list of elements of the alleged S'.

Step 1 (verifier). *On input $(\langle S, t\rangle, c)$:*

1. INDEX check: every entry of c appears in S.
2. DISTINCT check: the entries of c are pairwise distinct.
3. SUM check: compute $\sum_j a_{i_j}$ and compare with t.
4. ACCEPT iff all three checks pass.

Step 2 (running time). • Step 1: $O(m \cdot |S|)$.

• Step 2: $O(m^2)$.

• Step 3: $O(m \cdot |t|)$.

Total: $O(m\,|S| + m\,|t|) = \text{poly}(n)$.

Step 3. V is polynomial-time. Therefore SUBSETSUM $\in$ NP. ■

6. Hardness inheritance via reductions

Template — Inheritance proof from a reduction $A \leq B$

Step 1. State a precise definition of the reduction notion you are using ($\leq_p$, $\leq_m$, ...).

Step 2. Prove a one-step inheritance lemma: "If $A \leq B$ and A is `hard`, then B is `hard`." Use *composition*.

Step 3. Apply the lemma iteratively along a given chain of reductions.

Worked example — If $A \leq_m B$ is a (computable) many-one reduction and A is undecidable, prove that B is undecidable. Then apply the lemma to a chain $L_1 \leq_m L_2 \leq_m L_3 \leq_m L_4$ with L_1 undecidable to derive that L_4 is undecidable.

Step 1 (definition). $A \leq_m B$ means: there is a total computable function $f : \Sigma^* \to \Sigma^*$ such that for every x, $x \in A \Leftrightarrow f(x) \in B$.

Step 2 (one-step inheritance). **Claim.** If $A \leq_m B$ via f and A is undecidable, then B is undecidable.

Proof. Suppose, for contradiction, B is decidable; let H_B be its decider. Define H_A: *on input x, compute $f(x)$ then run H_B on $f(x)$, output H_B's verdict.* By the equivalence in Step 1, H_A decides A. But A is undecidable. Contradiction.

Step 3 (chain). Suppose L_1 is undecidable and $L_1 \leq_m L_2 \leq_m L_3 \leq_m L_4$.

- L_1 undecidable and $L_1 \leq_m L_2$, so by Step 2, L_2 is undecidable.
- L_2 undecidable and $L_2 \leq_m L_3$, so L_3 is undecidable.
- L_3 undecidable and $L_3 \leq_m L_4$, so L_4 is undecidable. ■

7. Defining a class and proving a relationship

Template — Define class, define class, prove inclusion

Step 1. Formal definition of class $\mathcal{C}_1$.

Step 2. Formal definition of class $\mathcal{C}_2$.

Step 3. Prove $\mathcal{C}_1 \subseteq \mathcal{C}_2$ by giving, for every $L \in \mathcal{C}_1$, an explicit witness machine in $\mathcal{C}_2$ that decides L.

Worked example — Define the class of languages recognised by deterministic finite automata (DFA-REC) and the class of languages decided by Turing machines (TM-DEC). Prove DFA-REC $\subseteq$ TM-DEC.

Step 1. DFA-REC $= \{L : \text{some DFA } D \text{ has } L(D) = L\}$. Equivalently, $L \in$ DFA-REC iff there is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ such that for every $w \in \Sigma^*$, the trajectory $\hat{\delta}(q_0, w)$ lands in F iff $w \in L$.

Step 2. TM-DEC $= \{L : \text{some TM } M \text{ decides } L\}$. Equivalently, there is a deterministic TM M such that $M(x)$ halts in q_{accept} iff $x \in L$, and halts in q_{reject} otherwise.

Step 3 (inclusion). Take any $L \in$ DFA-REC and let $D = (Q, \Sigma, \delta, q_0, F)$ recognise it. Construct a single-tape TM M_D:
On input x:

1. Set internal state to q_0 and head position to the leftmost cell.
2. Scan x left to right. At each step, read the cell, apply δ, and update the internal state.
3. After reading the entire string, ACCEPT iff the internal state is in F; else REJECT.

M_D always halts (after exactly $|x|$ scans), and accepts iff D accepts. So $L = L(M_D) \in$ TM-DEC. Therefore DFA-REC $\subseteq$ TM-DEC. ■

Three reminders.

1. Every TM you introduce must be defined as "On input ...:" followed by numbered actions.
2. Every reduction or contradiction proof closes with *"Therefore L is ... ■"*.
3. Quote the named result you are using (Rice's theorem, $\leq_m$ composition, pumping lemma, ...) at the moment you use it.